

CI/CD Pipeline with Azure DevOps, GitOps, and AKS

Automating Software Delivery with Cloud-Native Tools

Group 29

Group Members:

Wisdom Chiemela Uwaga
Bolarinwa Gbenga Akinde
Ayobami Edun
Osahon Igbogbo
Emmanuel Ohuoha

Module: Cloud Engineering

Date: May 2026

Institution: [Your Institution Name]

Table of Contents

1. Project Overview

1.1 What This Project Does

This project implements a production-grade CI/CD pipeline using Azure DevOps, GitOps (ArgoCD), and Azure Kubernetes Service (AKS). When a developer pushes code to GitHub, the pipeline automatically runs security scans (secrets, IaC misconfigurations, SAST, and container vulnerabilities), builds Docker images for three microservices, pushes them to Azure Container Registry, updates Kubernetes manifests in GitHub, and ArgoCD syncs the cluster — all without human intervention.

1.2 The Three Microservices

Three independent REST APIs demonstrating polyglot deployment:

- Python App (Flask/Gunicorn) — port 5000 — endpoints: /, /health, /api/data
- Node.js App (Express) — port 3000 — endpoints: /, /health, /api/data
- .NET App (ASP.NET Core) — port 8080 — endpoints: /, /health, /api/data

Each service runs with 2 replicas for high availability.

1.3 Project Objectives

- Automate the full software delivery lifecycle from code commit to running application
- Implement DevSecOps: security gates that block deployment if vulnerabilities are found
- Demonstrate GitOps: GitHub as the single source of truth; the cluster self-heals to match Git
- Apply production-grade Kubernetes security hardening
- Provide full observability: metrics, logs, and alerting via Azure Monitor

2. Technology Stack

Tool	Role	Why Chosen
Azure Kubernetes Service (AKS)	Container orchestration — runs all three microservices	Required by project brief; only platform supporting ArgoCD natively with full Kubernetes control
Azure DevOps Pipelines	CI/CD automation — builds, scans, and deploys	Required by project brief; native Azure integration for ACR, Key Vault, and AKS
Azure Container Registry (ACR)	Private Docker image registry (acrgitopsproject)	Native Azure integration; images stay within Azure network; attached to AKS cluster
ArgoCD	GitOps engine — watches GitHub and syncs the cluster	True GitOps pull model; self-healing (selfHeal:true); visual dashboard; referenced in project guide
Azure Key Vault	Secret management (kv-gitops-project)	Secrets never in code; access-controlled by RBAC; every access logged; less than \$1/month
Docker	Application containerisation	Industry standard; identical behaviour across environments

Gitleaks v8.18.4	Secrets scanner — blocks committed passwords	Prevents credential leakage before code reaches registry
Checkov 3.2.0	IaC scanner — audits Kubernetes manifests	Catches misconfigurations (no resource limits, privileged containers) before deploy
Semgrep 1.72.0	SAST — scans application source code	Language-aware pattern matching for insecure code patterns
Trivy 0.50.4	Container image CVE scanner	Scans built image for known vulnerabilities; blocks CRITICAL/HIGH before push to ACR
GitHub	Git repository — source of truth for code and config	ArgoCD polls this; all infrastructure state stored here; free for public repos

3. System Architecture

3.1 High-Level Flow

Developer commits code → GitHub → Azure DevOps CI Pipeline (Gate 1: Gitleaks → Gate 2: Checkov → Gate 3: Semgrep → Build 3 images in parallel → Trivy scan each → Push to ACR) → Azure DevOps CD Pipeline (Fetch secrets from Key Vault → Update image tags in GitHub → Trigger ArgoCD sync) → ArgoCD (detects Git change → applies to AKS) → AKS Cluster (3 nodes, microservices namespace) → 3 running services accessible via LoadBalancer IPs.

[SCREENSHOT PLACEHOLDER — Insert architecture diagram here]

3.2 Repository Structure

- microservices/ — Application source code (python-app, nodejs-app, dotnet-app), each with Dockerfile
- k8s/ — All Kubernetes manifests (deployments, services, RBAC, network policies, quotas, namespaces)
- argocd/ — ArgoCD Application and AppProject definitions
- azure-pipelines/ — CI pipeline (ci-pipeline.yml) and CD pipeline (cd-pipeline.yml)
- monitoring/ — Alert setup script and KQL query library
- scripts/ — Utility scripts (non-prod shutdown/start)
- docs/ — All documentation (threat model, trade-off matrices, DR plan, cost estimate, setup guide)

3.3 Trust Boundaries

The architecture enforces six trust boundaries:

1. Internet → Load Balancer
2. Load Balancer → Pod
3. Pod → Pod (blocked by NetworkPolicy — default-deny)
4. Pod → Kubernetes API (blocked by ServiceAccount restriction)

5. Pipeline → ACR (authenticated via Service Principal)
6. Pipeline → Key Vault (authenticated via Service Connection RBAC)

4. Infrastructure Setup

The infrastructure is created using Azure CLI commands documented in docs/setup-guide.md. Key steps:

7. Resource Group — rg-gitops-project in East US
8. Azure Container Registry — acrgitopsproject (Standard SKU, admin disabled, attached to AKS)
9. AKS Cluster — aks-gitops-cluster, 3 nodes (Standard_B2s), Container Insights enabled, ACR attached
10. Azure Key Vault — kv-gitops-project, stores argocd-admin-password and github-pat
11. ArgoCD — Installed into argocd namespace, exposed via LoadBalancer, password stored in Key Vault
12. Azure DevOps — Service connection to Azure subscription, pipeline variables for ACR and Key Vault name

[SCREENSHOT PLACEHOLDER — Azure Portal showing AKS cluster with 3 Ready nodes]

[SCREENSHOT PLACEHOLDER — Azure Portal showing Key Vault with two secrets]

5. Application Code and Dockerfiles

5.1 Python Application (Flask)

The Python app uses Flask with Gunicorn as the production WSGI server. The Dockerfile uses a multi-stage build: builder stage installs dependencies from requirements.txt, the final stage copies only the installed packages. The application runs as non-root user appuser (UID 1001). Key security measures: --tmp-dir /tmp for Gunicorn (required with readOnlyRootFilesystem), no secrets in image.

5.2 Node.js Application (Express)

The Node.js app uses Express.js. The Dockerfile uses node:20-alpine (minimal attack surface). Dependencies installed with npm ci --only=production (no dev dependencies in production). Runs as the built-in node user (UID 1000). Package versions pinned without caret (^) to prevent unexpected updates.

5.3 .NET Application (ASP.NET Core)

The .NET app uses ASP.NET Core Minimal API. Two-stage build: sdk image for compilation, runtime image for execution (smaller final image). Runs as non-root appuser (UID 1001). DOTNET_CLI_TELEMETRY_OPTOUT=1 set to disable telemetry data transmission.

5.4 Why Three Languages?

Using three different programming languages in three different Dockerfiles demonstrates that the pipeline is language-agnostic. The same CI/CD process — scan, build, scan, push — works regardless of whether the application is Python, JavaScript, or C#. This is a key property of a production CI/CD system.

6. Kubernetes Configuration

6.1 Namespace

All application resources are isolated in the microservices namespace. This provides logical separation from ArgoCD (argocd namespace) and kube-system components.

6.2 Deployments

Each deployment specifies:

- replicas: 2 — high availability; one pod can fail without service disruption
- Image tag replaced by CD pipeline with build-specific ID (never :latest)
- Resource requests and limits (enforced by LimitRange)
- Full security context (see Section 7)

6.3 Services

Three LoadBalancer services expose each application externally on port 80, forwarding to the container port. LoadBalancer type provisions an Azure public IP automatically.

6.4 Resource Quota

A ResourceQuota limits the entire microservices namespace to: 1500m CPU requests, 3000m CPU limits, 768Mi memory requests, 1536Mi memory limits, 20 pods maximum. A LimitRange sets default values so pods without explicit limits still get bounded. This prevents a single misbehaving pod from consuming all cluster resources.

[SCREENSHOT PLACEHOLDER — `kubectl get pods -n microservices showing all pods Running`]

[SCREENSHOT PLACEHOLDER — `kubectl get svc -n microservices showing external IPs`]

7. Security Implementation

7.1 Container Security Context

Every container in every deployment applies the following security context:

- runAsNonRoot: true — Kubernetes rejects the pod if the image tries to run as root
- runAsUser: 1001 — Explicit UID (1000 for Node.js which uses the built-in node user)
- allowPrivilegeEscalation: false — The process cannot gain more OS permissions than it started with
- readOnlyRootFilesystem: true — Nothing can write to the container filesystem; malware cannot persist
- capabilities: drop ALL — All Linux kernel capabilities removed; minimum OS access

- `seccompProfile: RuntimeDefault` — The Linux kernel blocks 45+ dangerous system calls at the kernel level

7.2 Kubernetes RBAC

Each application has a dedicated ServiceAccount (`python-app-sa`, `nodejs-app-sa`, `dotnet-app-sa`) with `automountServiceAccountToken: false`. This means application pods cannot call the Kubernetes API server — a compromised pod cannot enumerate other resources, create new pods, or escalate within the cluster.

7.3 Network Policies

A default-deny-all NetworkPolicy blocks all pod-to-pod communication by default. Individual allow policies then open only what is needed:

- DNS egress (port 53 UDP/TCP) — all pods need DNS to resolve service names
- Ingress on each service port (5000, 3000, 8080) from any source

Even if one pod is compromised, it cannot reach any other pod in the cluster.

7.4 Pipeline Security Gates

Four security gates run before any image reaches ACR:

13. Gitleaks — scans all committed files for secrets, API keys, passwords
14. Checkov — audits Kubernetes YAML manifests for misconfigurations
15. Semgrep — SAST scan of application source code
16. Trivy — CVE scan of the built Docker image

Any CRITICAL or HIGH finding causes the pipeline to fail and no image is pushed.

7.5 Secret Management

The ArgoCD admin password and GitHub Personal Access Token are stored in Azure Key Vault. The CD pipeline fetches them at runtime using the `AzureKeyVault@2` task. They never appear in: pipeline YAML, pipeline variables, application code, Kubernetes manifests, or Git history.

[SCREENSHOT PLACEHOLDER — Azure DevOps pipeline showing all security scan stages passing]

8. CI/CD Pipeline

8.1 CI Pipeline (ci-pipeline.yml)

Trigger: push to main branch or pull request.

Stages (in order):

17. SecretScan — Gitleaks v8.18.4 scans for committed secrets. Fail = pipeline stops, no build.
18. IaCScan — Checkov 3.2.0 scans k8s/ manifests. Fail on CRITICAL/HIGH = pipeline stops.
19. SAST — Semgrep 1.72.0 scans application source code. Fail = pipeline stops.
20. Build_Python, Build_NodeJS, Build_DotNet — Run in PARALLEL. Each: docker build → Trivy scan → push to ACR with tag `Build.BuildId`. Never pushes `:latest`.

Key design decisions:

- SARIF output from all scanners published as pipeline artifacts (evidence of scanning)
- Parallel build stages reduce pipeline duration from ~9 minutes to ~4 minutes
- Build.BuildId tag links every running pod back to a specific pipeline run

8.2 CD Pipeline (cd-pipeline.yml)

Trigger: CI pipeline completion (success only) OR manual trigger.

Stages (in order):

21. FetchSecrets — AzureKeyVault@2 retrieves argocd-admin-password and github-pat from Key Vault.
22. UpdateManifests — sed replaces image tag in all three deployment YAMLs. Git commits with [skip ci] to prevent infinite pipeline loop. Pushes using Key Vault PAT.
23. SyncArgoCD — Installs argocd CLI v2.10.3. Logs in with --grpc-web. Runs argocd app sync microservices-app. Waits up to 180 seconds for all pods to be healthy. Pipeline fails if sync fails.

[SCREENSHOT PLACEHOLDER — CD pipeline stages all green with sync successful]

9. GitOps with ArgoCD

9.1 How ArgoCD Works

ArgoCD runs inside the AKS cluster and continuously polls the GitHub repository every 3 minutes. When it detects a difference between what GitHub says (desired state) and what is running in the cluster (actual state), it automatically applies the change. This is the GitOps pull model — the cluster pulls its configuration from Git rather than having a pipeline push to it.

9.2 ArgoCD Application Definition

The application.yaml file tells ArgoCD:

- Which GitHub repository to watch
- Which directory (k8s/) contains the Kubernetes manifests
- Which namespace to deploy into (microservices)
- automated.prune: true — deletes resources that are removed from Git
- automated.selfHeal: true — reverts any manual changes to the cluster

9.3 ArgoCD AppProject

The project.yaml scopes ArgoCD's permissions to the microservices namespace only. Even if the ArgoCD admin account were compromised, it could not deploy resources to kube-system, argocd, or any other namespace.

9.4 Self-Healing Demonstration

If a team member runs 'kubectl delete deployment python-app -n microservices', ArgoCD detects the drift within 3 minutes and recreates the deployment from the Git-stored manifest. This makes the cluster configuration tamper-resistant.

[SCREENSHOT PLACEHOLDER — ArgoCD UI showing microservices-app Synced + Healthy]

10. Monitoring and Observability

10.1 What We Monitor

Six automated alert rules created by monitoring/setup-alerts.sh:

- 24. Pod Restart Count > 3 in 5 minutes — CrashLoopBackOff detection
- 25. Node CPU > 80% — Early warning before pod throttling
- 26. Node Memory > 85% — Warning before OOMKill terminates pods
- 27. Container CPU throttled > 90% — Performance degradation alert
- 28. Container Memory > 90% of limit — OOMKill imminent
- 29. ArgoCD OutOfSync > 10 minutes — Silent deployment failure detection

10.2 Log Analytics KQL Queries

Nine pre-built KQL queries documented in monitoring/log-queries.md cover: pod restart investigation, container log tailing, HTTP 5xx error analysis, node resource pressure, ArgoCD sync history, ImagePullBackOff diagnosis, full namespace event timeline, deployment audit trail, and security (unexpected processes).

10.3 Access

- AKS Insights — pod, node, and container metrics — Azure Portal > AKS cluster > Insights
- Log Analytics Workspace — raw logs via KQL — Azure Portal > Log Analytics Workspace > Logs
- Azure Monitor Alerts — alert history and current firing alerts — Azure Portal > Monitor > Alerts

[SCREENSHOT PLACEHOLDER — Azure Monitor showing alert rules]

[SCREENSHOT PLACEHOLDER — Log Analytics KQL query results]

11. Disaster Recovery

11.1 DR Metrics

Metric	Value	Rationale
RTO	2 hours	Time to rebuild AKS cluster, reinstall ArgoCD, re-sync all apps from Git
RPO	0 minutes	All services are stateless. Git is the source of truth and is hosted on GitHub, unaffected by AKS failure.
Recovery region	East US (same)	Project scope: single-region. Cross-region DR is a

		documented residual risk.
--	--	---------------------------

11.2 Recovery Scenarios

30. Single node failure — Kubernetes reschedules automatically; manual drain if node does not recover in 10 minutes
31. ArgoCD crash — kubectl rollout restart the three ArgoCD deployments
32. Full cluster loss — Rebuild from IaC: new AKS cluster → reinstall ArgoCD → reconnect GitHub → apply ArgoCD application → cluster self-heals from Git (estimated 45-90 minutes)
33. DR test — Required before first demo; procedure documented in Section 4 of disaster-recovery.md

11.3 Why Zero Data Loss?

All three microservices are stateless REST APIs with no database. There is no application state to lose. The only state is: (a) the code, stored in GitHub, and (b) the infrastructure configuration, also stored in GitHub. A full cluster rebuild recreates the exact same running state from Git.

12. Key Architectural Decisions

Decision 1 — Container Orchestration: AKS vs Azure Container Apps vs App Service

AKS chosen: only option that supports ArgoCD natively, provides full control over NetworkPolicy and pod-level security, and is explicitly required by the project brief. Higher cost (~\$130/month vs ~\$45/month for Container Apps) is justified by production fidelity and learning outcome.

Decision 2 — GitOps Delivery: ArgoCD vs Flux vs Plain kubectl

ArgoCD chosen: true GitOps pull model with self-healing, visual UI for demonstrating sync status, and referenced in the project guide. Plain kubectl rejected (push model, no drift detection). Flux is a valid alternative but has higher learning curve and no UI.

Decision 3 — CI/CD Platform: Azure DevOps vs GitHub Actions vs Jenkins

Azure DevOps chosen: required by project brief, native Azure service connections, built-in Key Vault and ACR tasks. GitHub Actions noted as the preferred evolution path for a real production system (OIDC authentication would eliminate the Service Principal entirely).

Decision 4 — Secret Management: Azure Key Vault vs Pipeline Variables vs Hardcoded

Key Vault chosen: hardcoded secrets rejected outright (visible in Git history permanently). Pipeline variables visible to project admins with no audit log. Key Vault provides: RBAC-controlled access, every read logged, rotation without pipeline change, less than \$1/month cost.

Conclusion

This project demonstrates a complete, production-grade CI/CD pipeline that applies industry best practices across every layer of the stack:

- Security: four scan gates before deployment, container hardening, network isolation, secret management
- Automation: zero manual steps from code push to running application
- GitOps: cluster state is always traceable to a specific Git commit
- Observability: six automated alerts plus a library of KQL queries for troubleshooting
- Resilience: 2-hour RTO with zero data loss, documented and testable

The architecture is not merely functional — it is defensible. Every technology choice is documented with trade-off analysis. Every security control addresses a specific threat from the STRIDE model. Every alert covers a failure mode with a documented impact.

Group 29 — Cloud Engineering — May 2026